NVM2+ NVR2+ UV ENGINE

USER'S GUIDE

Version : 0.0

Date: 2021/11/1

REVISION HISTORY

Date	Revision	Changes and Additions	Page
2021/11/1	0.0	Preliminary Version 0.0	All

Content

1 OVERVIEW 6

2 UV ENGINE INTRODUCTION 6

3 UV ENGINE SPECIFICATION 6

3.1 Standard Test Conditions 6

3.2 Optical Performance 7

4 IMAGE SPEC 8

4.1 Image Size 8

4.2 UV Engine Dimension 9

4.3 Optical Perfomance Measurement 10

4.3.1 Power Meter and Sensor Instrument 10

4.3.2 Output Power Measurement 11

4.3.3 ANSI Uniformity Measurement 12

4.3.4 FO:FO Contrast Ratio Measurement 13

4.4 Image Focus Adjustment 13

4.4.1 NVR2 Plus Focus Adjustment 13

4.4.2 NVM2 Plus Focus Adjustment 15

5 OPERATION PROCEDURE 16

5.1 Power Requirement 16

5.2 Transfer Board I/F Introduction 16

5.2.1 Connector Information 17

5.2.1.1 12V Input Connector Pin Out (J47) 18

5.2.1.2 I^2C I/F Conncetor Pin Out (J50) 18

5.2.1.3 Signal Input Connector Pin Out (J46) 18

5.3 Power Up/Down Sequence and I/O Signal Type 19

5.4 UV Light Engine Turn ON/OFF 20

5.5 Indicator Light ON The Board 22

5.5.1 Transfer Board 22

5.5.2 Formatter Board 23

5.6 Suggestion Process Of UV Engine In Printing Application 24

5.7 UI Operation Introduction 26

5.8 I^2C Command Setting 27

5.8.1 LED Current Setting 27

5.8.2 LED Current Reading 27

5.8.3 LED Enable/Disable 28

5.8.4 LED Enable 28

5.8.5 Read LED Junction Temperature 28

5.8.6 Projector Image Filp setting 29

5.8.7 Projector Image Flip Reading 29
5.8.8 READ LIGHT SENSOR VALUE 30
5.8.9 PROJECTOR SOURCE AND TEST PATTERNS SELECTION 30
5.8.10 READ PROJECTOR SOURCE AND TEST PATTERNS SELECTION 31
5.9 CYPRESS API FUNCTION INtroduction 33
5.9.1 DEVICE-INDEPENDENT FUNCTIONS 33
5.9.1.1 CYGETDEVICEINFOVIDPID 33
5.9.1.2 CYOPEN 34
5.9.1.3 CYCLOSE 34
5.9.2 GPIO 35
5.9.2.1 CYSETGPIOVALUE 35
5.9.2.2 CYGETGPIOVALUE 35
5.9.2.3 FUNCTION IMPLEMENTATION(GPIO) 36
5.9.2.3.1 Porjector on/off (Set) 36
5.9.2.3.2 HOST_IRQ Signal (Get) 36
5.9.3 I2C 36
5.9.3.1 CYI2CWRITE 36
5.9.3.2 CYI2CREAD 38
5.9.4 SPI 38
5.9.4.1 CYSPIREADWRITE 39
5.10 Pixel Data Mixed with Mask File and Read Mask File from SPI Flash 40
5.10.1 PREREQUISITE 40
5.10.2 MIXED OPERATION WITH MASK FILE 40
5.10.3 MASK FILE READ FROM SPI FLASH 40
5.10.4 SPI FLASH DATA STRUCTURE FOR MASK FILE 41
5.10.5 Read Mask file steps 41
5.10.6 Read calibrated LED current and light sensor value 43
5.11 Focus Motor specification 43

6 MAX. THERMAL RECOMMENDATION 44

7 PACKING 45

8 TROUBLE SHOOTING 46

Caution

1. **DO NOT touch heatsinks.**

   The DMD heatsink is connected to DMD chip. If the DMD heatsink is hit or dropped, it may cause a abnormal projection image or DMD damaged.

2. **DO NOT touch circuit boards.**

   The circuit boards are ESD sensitive. Touch the circuit boards without ESD protection may cause electrical components on circuit boards damaged.

3. **Do follow the power up and power down sequence defined in page 12**

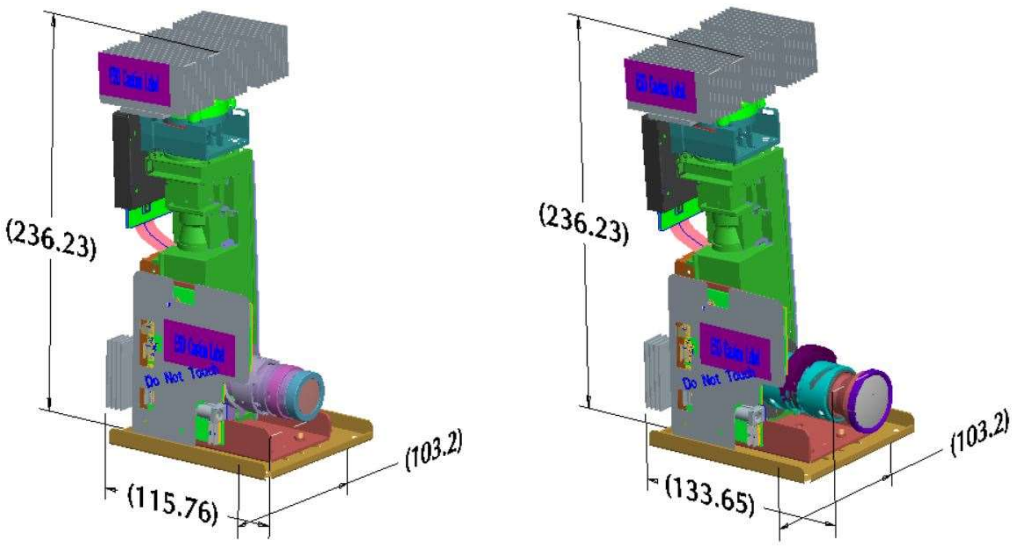
   Failure to adhere to the power up and power down procedures will cause the DMD permanent damage.

1 OVERVIEW

This user's guide presents an overview of NVM2 Plus and NVR2 Plus UV Light Engine and a general description of the main features and functions. It explains the first steps for the user to get started.

2 UV ENGINE INTRODUCTION

NVM2 Plus and NVR2 Plus UV Engine is configured based on UV LED and Texas Instruments' DMD. All glass lenses and optical coatings are optimized for 385nm and 405nm wavelength range of UV light. Besides, 0% offset optical architecture is designed for getting smaller distortion.



NVM2+ **NVR2+**

Figure 1. NVM2 Plus and NVR2 Plus UV Light Engine Dimension Overview

3 UV ENGINE SPECIFICATION

3.1 STANDARD TEST CONDITIONS :

Items	Condition
Warm-Up Period	5 minutes

LED Duty Cycle	100%
LED Junction Temperature	System needs to provide the cooling to LED module to make Tj below 125 °C
Ambient Light	0mW (Dark Room)
Ambient Temperature / Humidity	25 ± 2 °C , 60~70% RH
Input Signal	1920 x 1080 , 60Hz

[Note] The operation ambient temperature is 5~35°C for reference.

3.2 OPTICAL PERFORMANCE :

Parameter	Value
UV Output Power	(without uniformity MASK calibration) [NVM2 Plus, NVR2 Plus 405nm Engine] 1900 mW (typ.) [NVM2 Plus, NVR2 Plus 385nm Engine] 2000 mW (typ.) [NVR2 Plus, 365nm Engine] 850 mW (typ.) Operating LED current (max.) : 7A (DAC setting : 1023)
	(with uniformity MASK calibration) [NVM2 Plus, NVR2 Plus 405, 385nm Engine] 900 +/- 10 mW Operating current : Default setting
ANSI Uniformity	(without uniformity MASK calibration) ≧ 90%
	(with uniformity MASK calibration) ≧ 98%
FO:FO Contrast Ratio	(without uniformity MASK calibration) 1100 : 1 (typ.)
	(with uniformity MASK calibration) 1000 : 1 (typ.)
TV Distortion	≦ 1%
Image Size	Image Width ± 1 mm, Image Height ± 1 mm
Depth of Focus	Projection Distance ± 1 mm

4 IMAGE SPEC

4.1 IMAGE SIZE :

NVM2 Plus and NVR2 plus UV light engine supports projection image pixel size from 30um to 100um. The following table shows projection image information for reference.

Image Size	2.6	3.5	4.3	5.6	6.1	6.5	6.8	8.6	Inch
Image Width	57.6	76.8	96.0	124.8	134.4	144.0	149.8	192.0	mm
Image Height	32.4	43.2	54.0	70.2	75.6	81.0	84.2	108.0	mm
Image Diagonal	66.1	88.1	110.2	143.2	154.2	165.2	171.8	220.3	mm
Image Pixel Size	**30**	**40**	**50**	**65**	**70**	**75**	**80**	**100**	**um**
Image Resolution	847	635	508	391	363	339	326	254	ppi
NVR2 Projection Distance (Optical)	50.1	72.6	95.2	128.9	140.3	151.7	163.1	208.6	mm
NVR2 Projection Distance (Mechanical)	111.6	135.2	158.7	193.4	205.0	216.5	228.1	274.1	mm
NVM2 Projection Distance (Optical)	96.7	141.4	186.0	232.9	252.4	272.0	283.9	368.8	mm
NVM2 Projection Distance (Mechanical)	144.3	188.9	233.1	280.4	300.0	319.6	332.1	456.6	mm

[Note] Projection distance (optical) is defined as top of first lens to projection image. Projection distance (mechanical) is defined as guide pin of light engine to projection image, refer to Figure 2 and Figure 3.

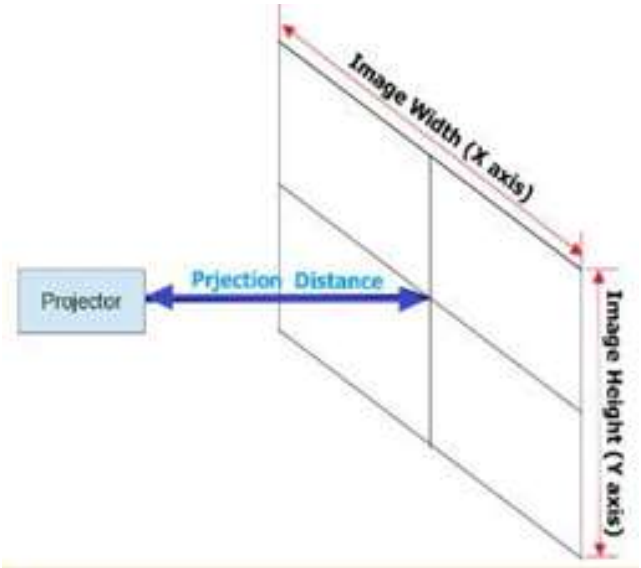


Figure 2. Image Definition

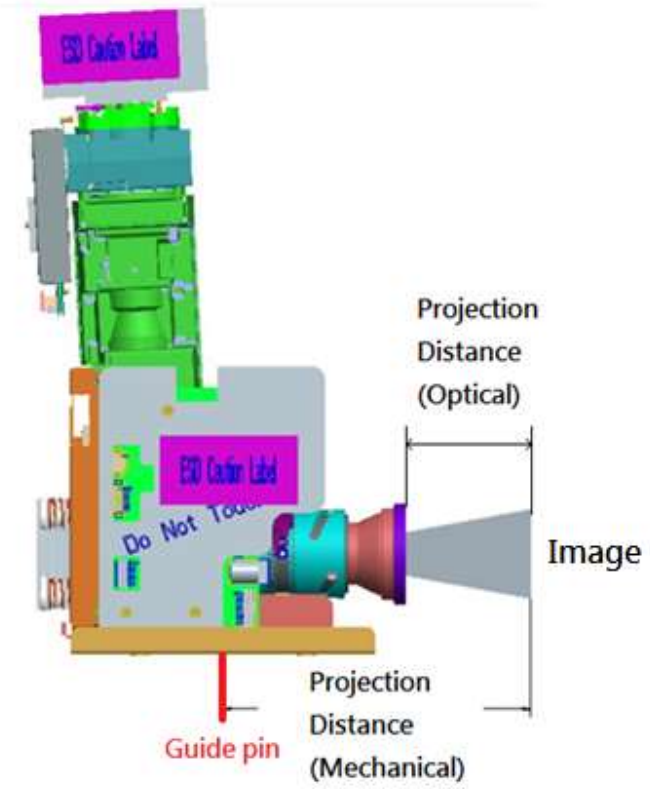


Figure 3. Projection Distance Definition

4.2 UV ENGINE DIMENSION :

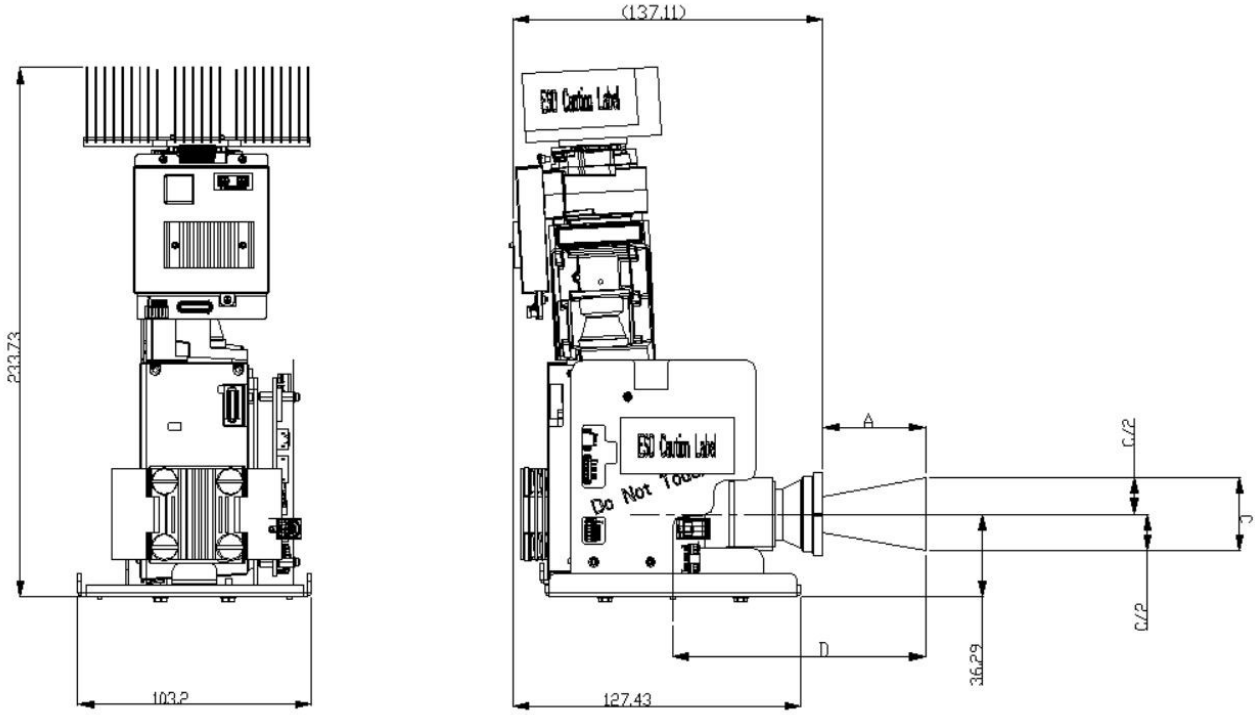


Figure 4. UV engine dimension

4.3 OPTICAL PERFORMANCE MEASUREMENT :

4.3.1 POWER METER AND SENSOR INSTRUMENT :

Power meter type : OPHIR StarLite

Power sensor type : OPHIR PD300RM-8W, with cosΘ angle correlation

OPHIR website as following link :

https://www.ophiropt.com/laser--measurement/laser-power-energy-meters/products/laser-power-meters/starlite

https://www.ophiropt.com/laser--measurement/laser-power-energy-meters/products/LED-Measurement/Irradiance-and-Dosage-sensors/PD300RM-8W

Figure 5. Instrument

4.3.2 OUTPUT POWER MEASUREMENT :

Step.1 : LED current should be set as following conditions before measurement :

Without uniformity mask calibration : max. LED driving current is 7A.

With uniformity mask calibration : LED driving current is default setting.

Step.2 : Set full white image, pixel size of projection image is 100 um (192.0 mm x 108.0 mm).

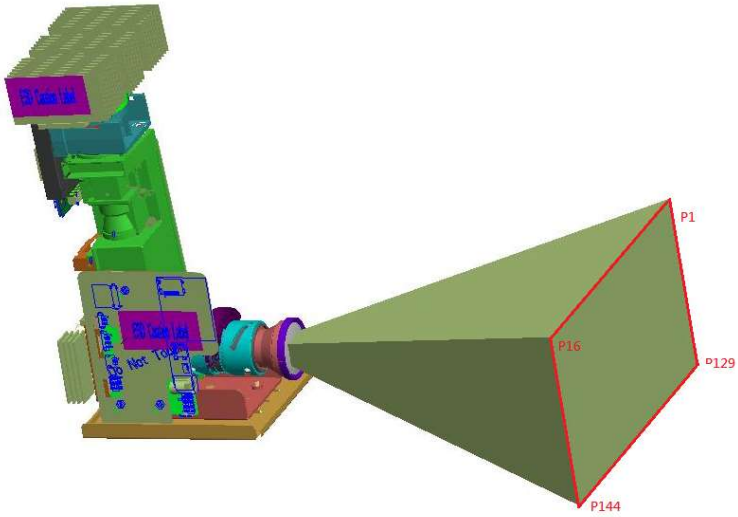
Step.3 : Measure the power density (mW/cm2) of every cell on full white pattern sequentially.

The full white pattern is defined with 144 cells, refer to Figure 6.

Step.4 : Summarize and calculate power density of every cell, refer to following method :

Total power density (unit: mW/cm2) = Summarize power density (Cell 1 + Cell 2 + ... + Cell 144)

Trans to total power (unit: mW) = (Total power density) x (0.502655) x (2.864789)



Position

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16
P17	P18	P19	P20	P21	P22	P23	P24	P25	P26	P27	P28	P29	P30	P31	P32
P33	P34	P35	P36	P37	P38	P39	P40	P41	P42	P43	P44	P45	P46	P47	P48
P49	P50	P51	P52	P53	P54	P55	P56	P57	P58	P59	P60	P61	P62	P63	P64
P65	P66	P67	P68	P69	P70	P71	P72	P73	P74	P75	P76	P77	P78	P79	P80
P81	P82	P83	P84	P85	P86	P87	P88	P89	P90	P91	P92	P93	P94	P95	P96
P97	P98	P99	P100	P101	P102	P103	P104	P105	P106	P107	P108	P109	P110	P111	P112
P113	P114	P115	P116	P117	P118	P119	P120	P121	P122	P123	P124	P125	P126	P127	P128
P129	P130	P131	P132	P133	P134	P135	P136	P137	P138	P139	P140	P141	P142	P143	P144

Figure 6. Image definition of 144 cells

4.3.3 ANSI UNIFORMITY MEASUREMENT :

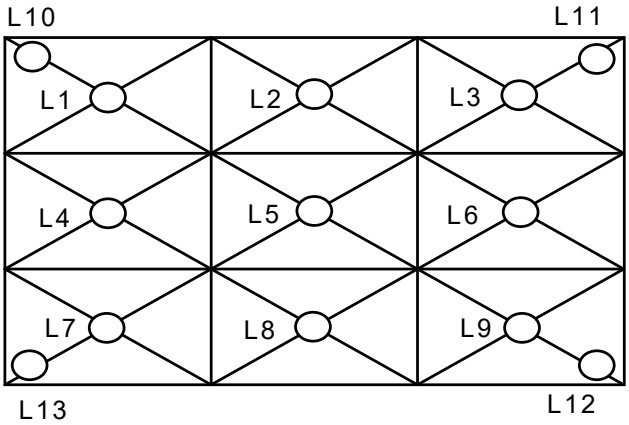
Step.1 : LED current should be set as following conditions before measurement :

Without uniformity mask calibration : max. LED driving current is 7A.

With uniformity mask calibration : LED driving current is default setting.

Step.2 : Set full white image, pixel size of projection image is 100 um (192.0 mm x 108.0 mm).

Step.3 : Measure power density (mW/cm2) of L1~L13 points sequentially defined as following picture :



The image four points L10~L13 are located at 10% of the distance from the corner itself to the center of measurement location L5.

Step.4 : Power uniformity is the min. output power (mW) of L1~L13 position divided by the output power (mW) of position L1~L9 on full white image, refer to following method :

ANSI Uniformity = ((min. of L1 to L13) – (avg. of L1 to L9)) / (avg. L1 to L9}

4.3.4 FO:FO CONTRAST RATIO MEASUREMENT :

Step.1 : LED current should be set as following condition before measurement :

Without uniformity mask calibration : max. LED driving current is 7A.

With uniformity mask calibration : LED driving current is default setting.

Step.2 : Set full white and black image, pixel size of projection image is 100 um (192.0 mm x 108.0 mm).

Step.3 : Measure power density (unit: mW/cm2) of central white and black image.

Step.4 : Calculate FO:FO contrast ratio, refer to following method :

FO:FO contrast ratio = power density of full white image / full black image at L5

4.4 IMAGE FOCUS ADJUSTMENT :

4.4.1 NVR2 PLUS FOCUS ADJUSTMENT :

Step.1 : Fix the optical (or mechanical) projection distance of UV engine, refer to table **4.1 IMGE SIZE.**

Step.2 : Align the position-1 and roller-1, turn front group of projection lens with P1 to P7 marks. These marks represent image pixel size 30~100um, front group should be adjusted correctly. The following table shows image pixel size and motor steps for reference.

Figure 7. Mark of front group

Mark on front group	Pixel size (unit: um)	Reference conversions for oblique groove movement step (unit: full step)
2	30	319
3	40	687
4	50	881
5	56	1022
6	65	1139
---	70	1189
---	75	1238
---	80	1264
7	100	1401

Step.3：Check sharpness of central image. If sharpness is blur, adjust rear group of projection lens to find the best sharpness.

Step.4：Check sharpness of image corners. If sharpness are blur, fine-tune front group of projection lens to find the best sharpness.

Step.5：Repeat step.3 and step.4, fine-tune front and rea groups to optimize the best sharpness of whole image.

Step.6：Fix the front and rear group, tight the two screws on projection lens, use screw driver with torque 3kgf*cm, refer to following picture:

Figure 7. Screws of front and rear projection lens

4.4.2 NVM2 PLUS FOCUS ADJUSTMENT :

Step.1 : Fix the projection distance of UV engine, refer to the table **4.1 IMGE SIZE**

Step.2 : Turn the projection lens to focus image sharpness.

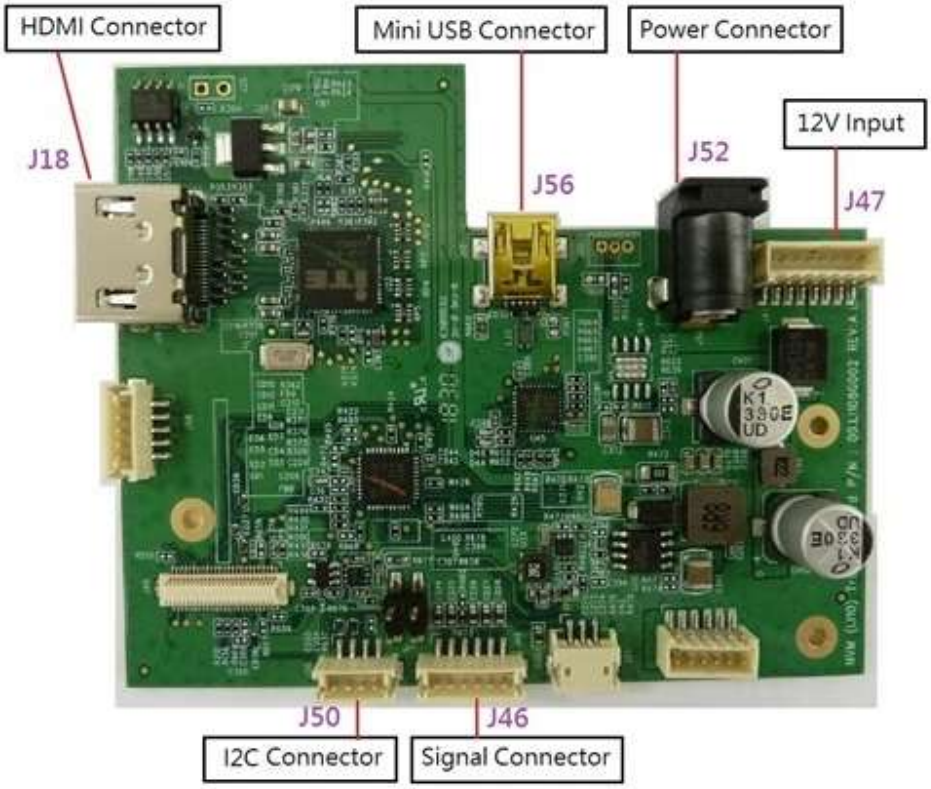
5 OPERATION PROCEDURE

Part	Qty	Picture	Assembly
Light Engine	1		
Transfer Board	1		

5.1 POWER REQUIREMENT

	Min.	Typical	Max.	Unit
Voltage	11.4	12	14	V
Power consumption	90			W

5.2 TRANSFER BOARD I/F INTRODUCTION

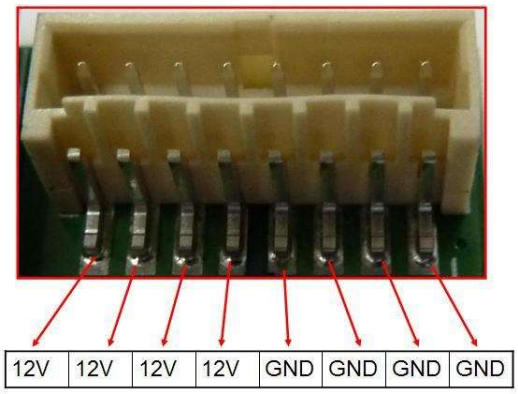


- Power Connector：For 12V/ 7.5A DC adaptor input (Φ2.0mm or Φ2.1mm type)
- 12V input: Location J47 for alternative12V power input（See 5.2.1 for details）
- Mini USB connector：Connect to computer for Light Engine Control (USB to I2C, USB to SPI)
- HDMI Connector：For 1080p (1920x1080 / 60Hz) video resolution input source
- I2C connector：Location J50 for I2C communication function（See 5.2.1 for details）
- Signal Connector：Location J46 for control signals in/out（See 5.2.1 for details）

5.2.1 CONNECTOR INFORMATION

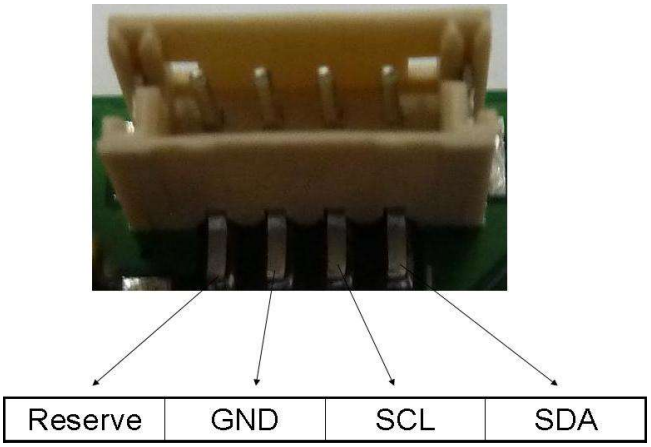
Location	Pin Number	Function	P/N	Vendor
J47	8	12V Input	TU1502WNV-08S	Tarng Yu
J50	4	I2C I/F	TU1501WNV-04S	Tarng Yu
J46	6	Singal Input	TU1501WNV-06S	Tarng Yu

5.2.1.1 12V INPUT CONNECTOR PIN OUT (J47)



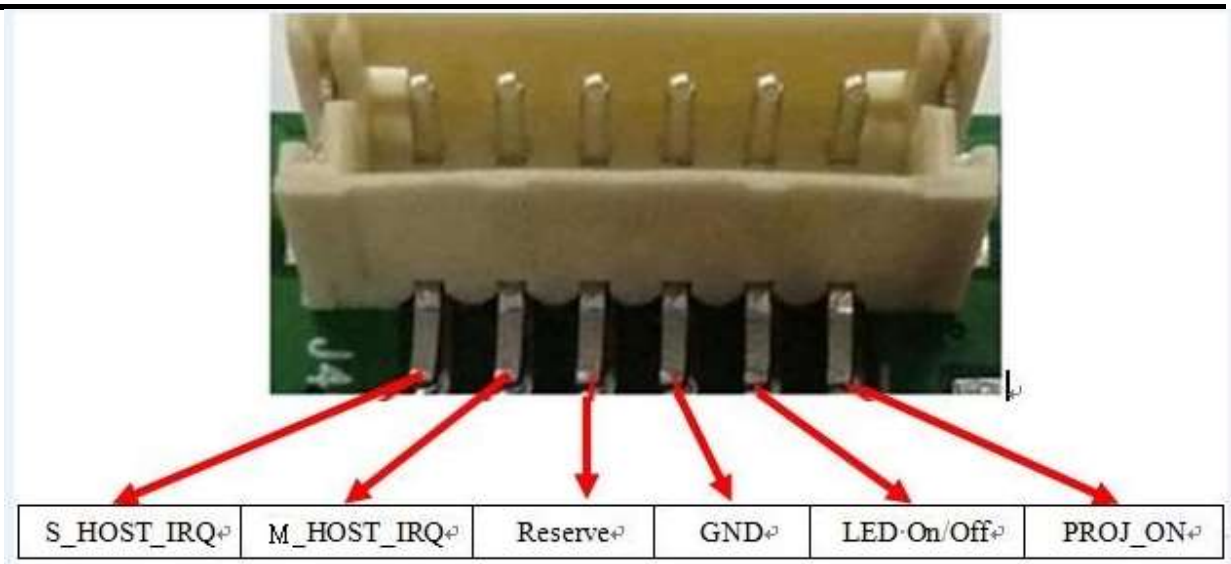
- Connector Information
  - Supplier: Tarng Yu Enterpries Co.,Ltd.
  - Type : TU1502WNV-08S
  - Website : http://www.tyu.com.tw/
- Suggestion for Housing type：TU1502HNO-08

5.2.1.2 I²C I/F CONNECTOR PIN OUT (J50)



- Connector Information
  - Supplier: Tarng Yu Enterpries Co.,Ltd.
  - Type : TU1501WNV-04S
  - Website : http://www.tyu.com.tw/
- Suggestion for Housing type：TU1501HNO-4

5.2.1.3 SIGNAL INPUT CONNECTOR PIN OUT (J46)



- Connector Information
  - Supplier: Tarng Yu Enterpries Co.,Ltd.
  - Type : TU1501WNV-06S
  - Website : http://www.tyu.com.tw/
- Suggestion for Housing type：TU1501HNO-6

Note：

- Please install the USB driver on the computer first if you need to access mini USB connector.
- USB driver download website :

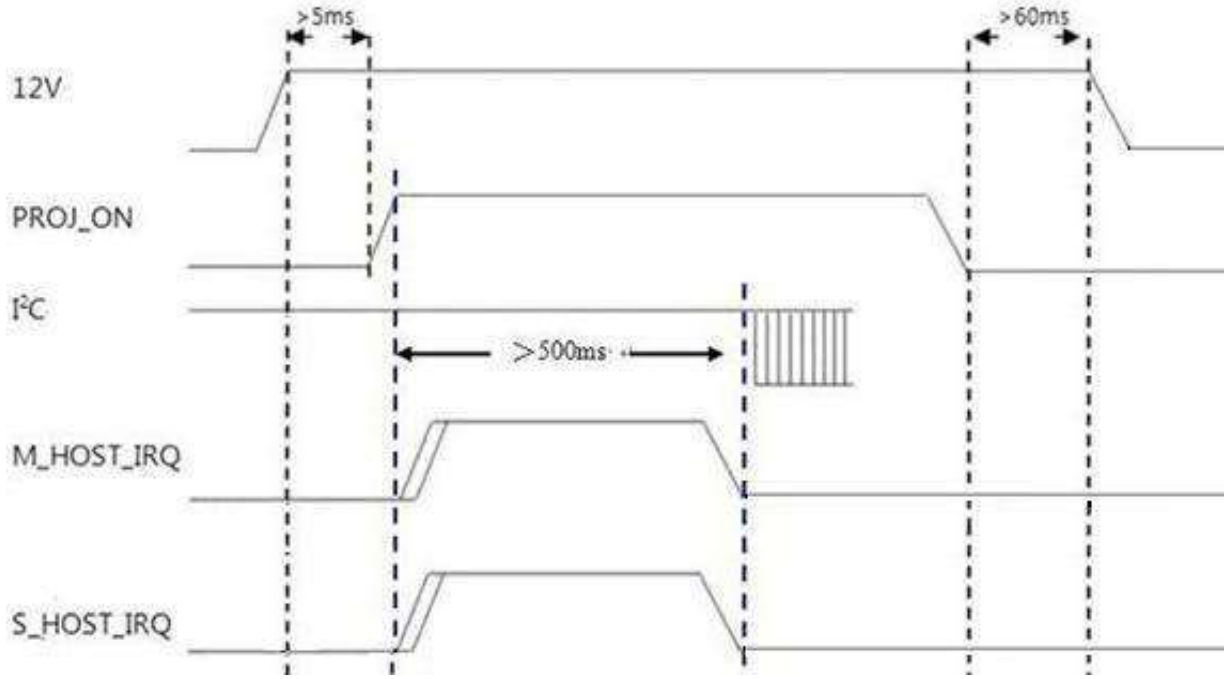
  http://www.cypress.com/documentation/software-and-drivers/usb-serial-software-development-kit

- Power input option：Either Location J52 or Location J47
- Light Engine control option：Either Location J56 Mini USB connector or Location J50 I^2C Interface.
- PROJ_ON control option： Location J46 or by USB I/F control.
- LED On/Off Pin can control by Controller, See chapter 5.7 UI Operation Introduction.

  Both I2C and GPIO LED On, the LED will On.

5.3 POWER UP/DOWN SEQUENCE AND I/O SIGNAL TYPE

- Power up and power down sequence



Note:

1) Please follow power up and power down sequence to avoid DMD damage.

2) Upon completion of initialization of light engine, the M_HOST_IRQ / S_HOST_IRQ will be set to a logic Low state to indicate the completion of initialization. In the initialization meantime, I2C access to ASIC should not start until M_HOST_IRQ / S_HOST_IRQ goes low.

- I/O Signal type definition

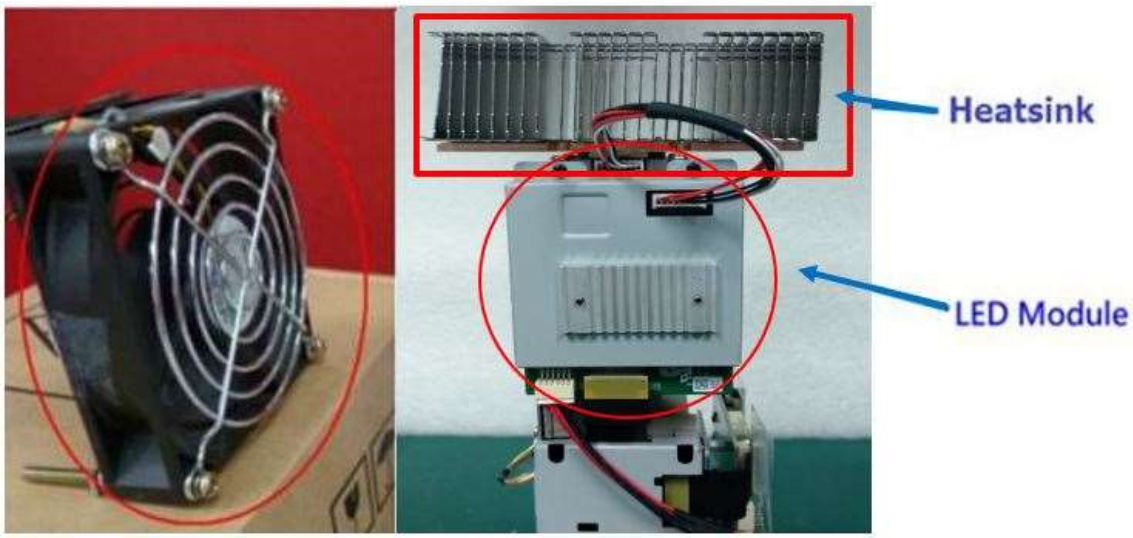
Pin Name	Voltage Level
PROJ_ON	3.3V LVCMOS
SCL	3.3V LVCMOS
SDA	3.3V LVCMOS
LED On / Off	3.3V LVCMOS
M_HOST_IRQ	3.3V LVCMOS
S_HOST_IRQ	3.3V LVCMOS

5.4 LIGHT ENGINE TURNED ON/OFF

(1) Before Turn ON the Light Engine, PLEASE MAKE SURE:

(a) DO NOT bridge the Heat sink to other objects with difference electric potential of LED Cathode.

(b) **PLEASE DO** provide the sufficient air flow (>15CFM) to LED heat-sink to avoid damaging the LED and PCBA.



(2) Turn on the Light Engine

Step 1: Plug-in Mini USB cable and HDMI cable first and then turn on 12V DC Adapter Power (From Location J52 or Location J47).

Step 2: Please choose either method 1 or method 2 to turn on light engine.

Method 1: Set Pin "PROJ_ON" signal of J46 connector to high level.

Method 2: Using UI software to turn on Light Engine via mini USB connector. (Please see "5.7 UI Operation Introduction'' section for details.)

Note: The LED is disabled if there is no video signal input to projector, even if turn on process is finish.

(3) Set LED ON/OFF

Method 1：Set I2C command and J46 pin name "LED On/Off"

Control true table as below:

LED On / Off on J46	LED On/Off of I2C command	LED Status
High	1	ON
High	0	OFF (Default)
Low	1	OFF
Low	0	OFF

Note:

1. For J46 pin LED On / Off, please see "5.2.1.3 Signal Input Connector Pin Out (J46)" for details.

2. LED default status is OFF. The default state of LED On / Off on J46 is “High”. The default setting of LED On / Off of I2C command is “0”.

Method 2：Set LED current. (Please refer to “5.8 I^2C Command set” for details).

(4) Turn off the Light Engine

Step1: Please choose either method 1 or method 2 to turn off light engine.

Method 1: Set Pin “PROJ_ON” signal of J46 connector to low level.

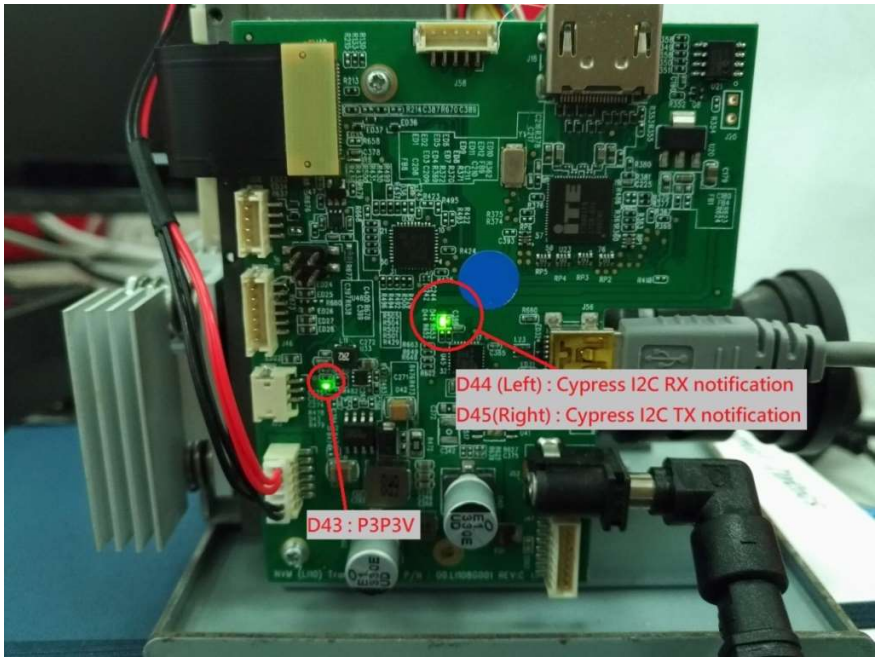
Method 2: Use UI software to turn off light engine. (Please see “5.7 UI Operation Introduction'' section for details)

Step2: Turn off 12V DC Adapter Power (Location J52 or Location J47).

Note：Please follow power up and power down sequence accordingly; wrong operation procedure will cause DMD damaged.

5.5 INDICATOR LIGHT ON THE BOARD

5.5.1 TRANSFER BOARD

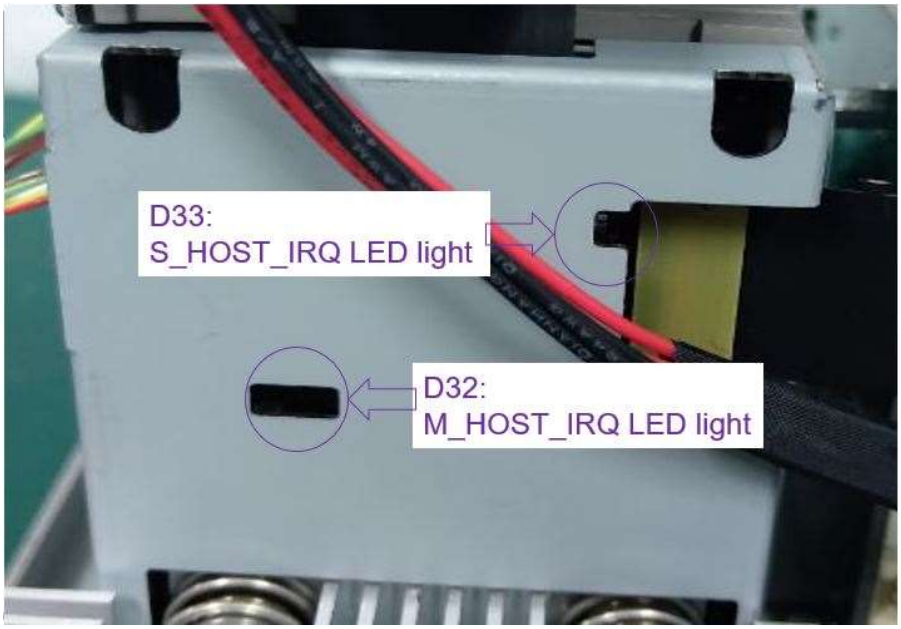


1. D43 (P3P3V) :
   Light up : P3P3V is enable after projector on.
   Light goes out : P3P3V is disable after projector off.

2. D44 (Cypress I2C RX)
   Light up : Cypress chip receives data via I2C, and if no data received within ten seconds, the light will go out.

3. D45 (Cypress I2C TX)
   It will only light up when Cypress chip transmits data via I2C. So, if you transmit datum continuously, the light will be flashing.

5.5.2 FORMATTER BOARD

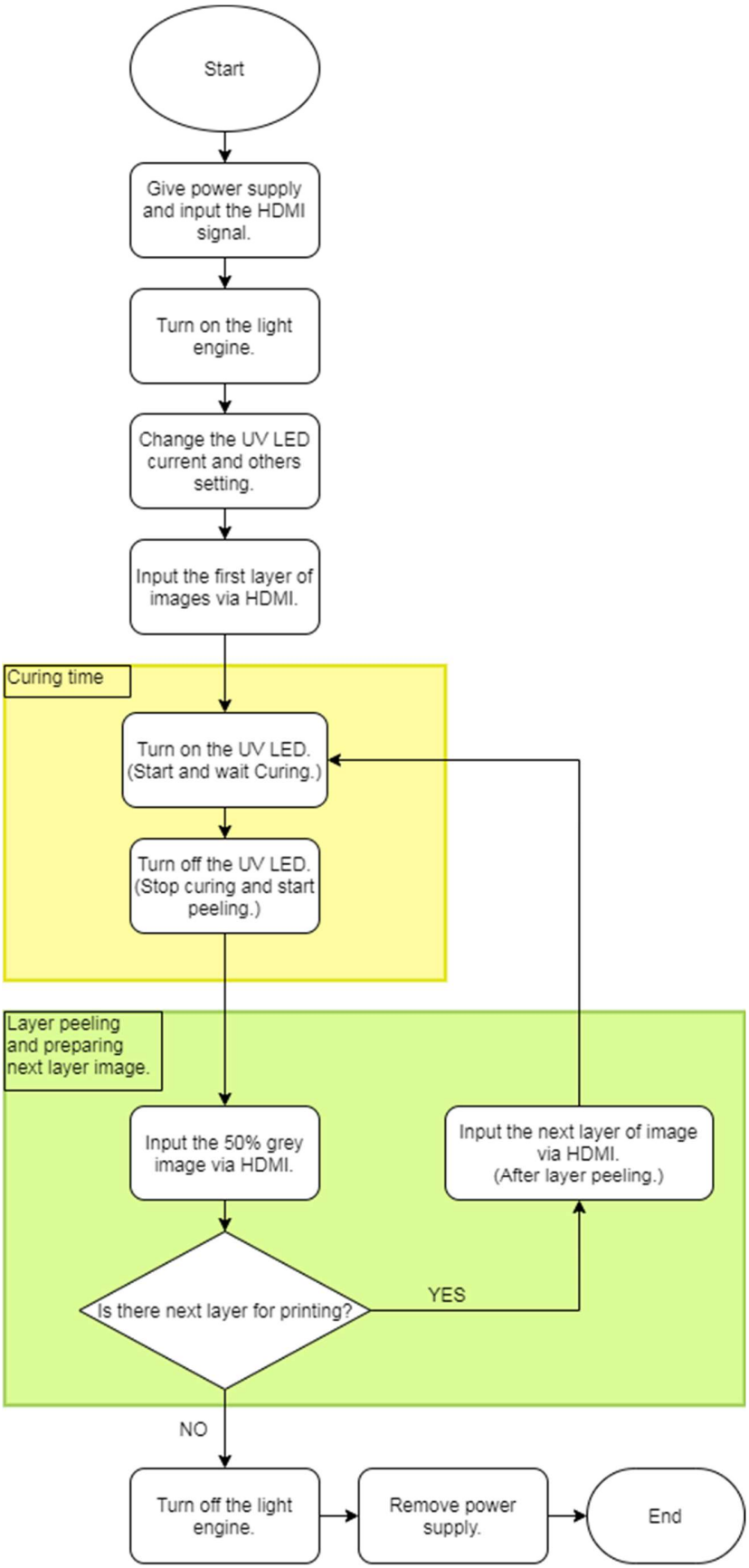


1. D32 (DLPC3439_Master HOST_IRQ) :
   Light up : DLPC3439 master has completed boot up and initialization after projector on.
   Light goes out : Projector off or DLPC3439 master is in boot up and initialization state after projector on.

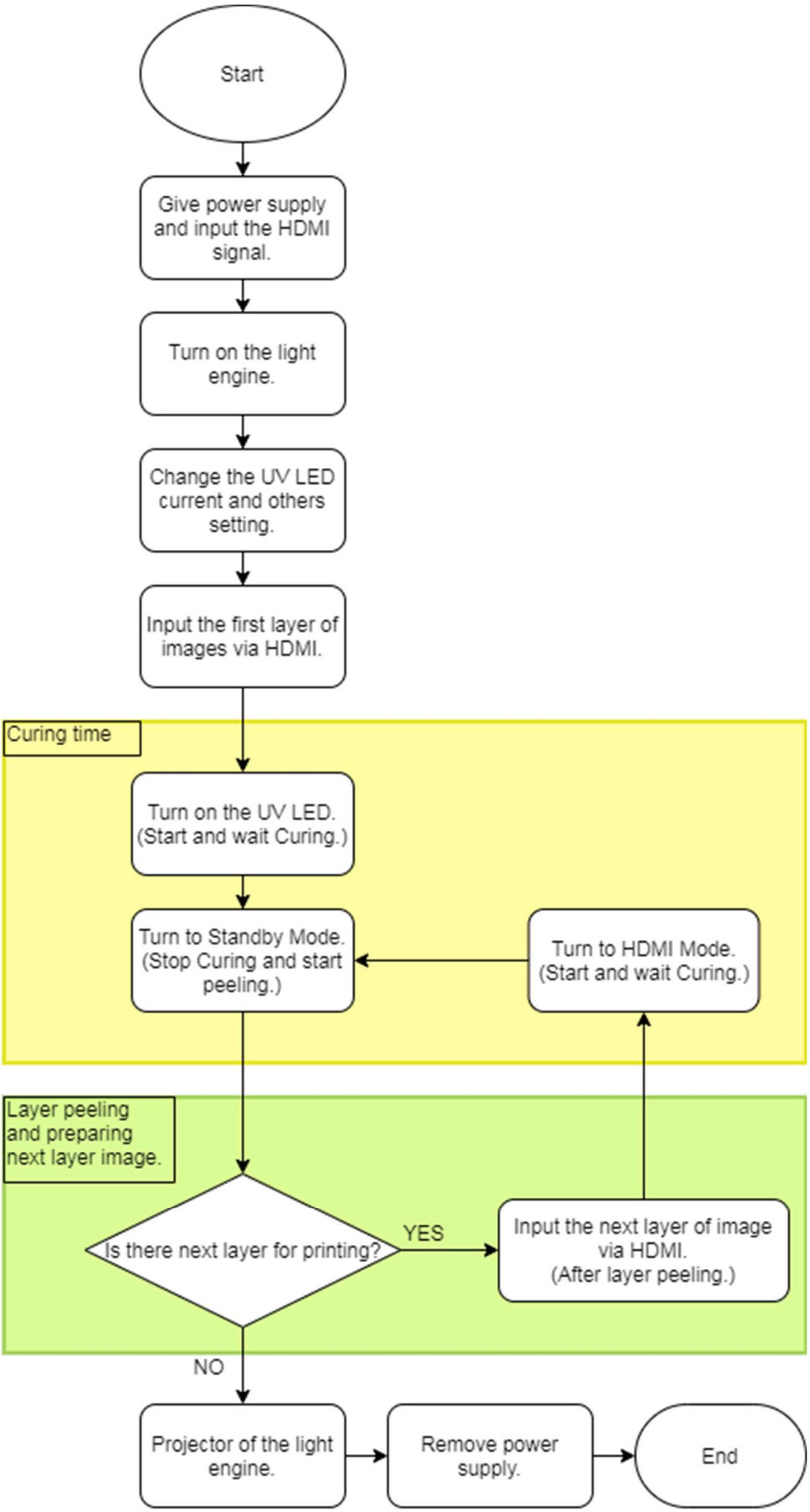
2. D33 (DLPC3439_Slave HOST_IRQ) :
   Light up : DLPC3439 slave has completed boot up and initialization after projector on.
   Light goes out : Projector off or DLPC3439 slave is in boot up and initialization state after projector on.

5.6 SUGGESTION PROCESS OF UV ENGINE IN PRINTING APPLICATION

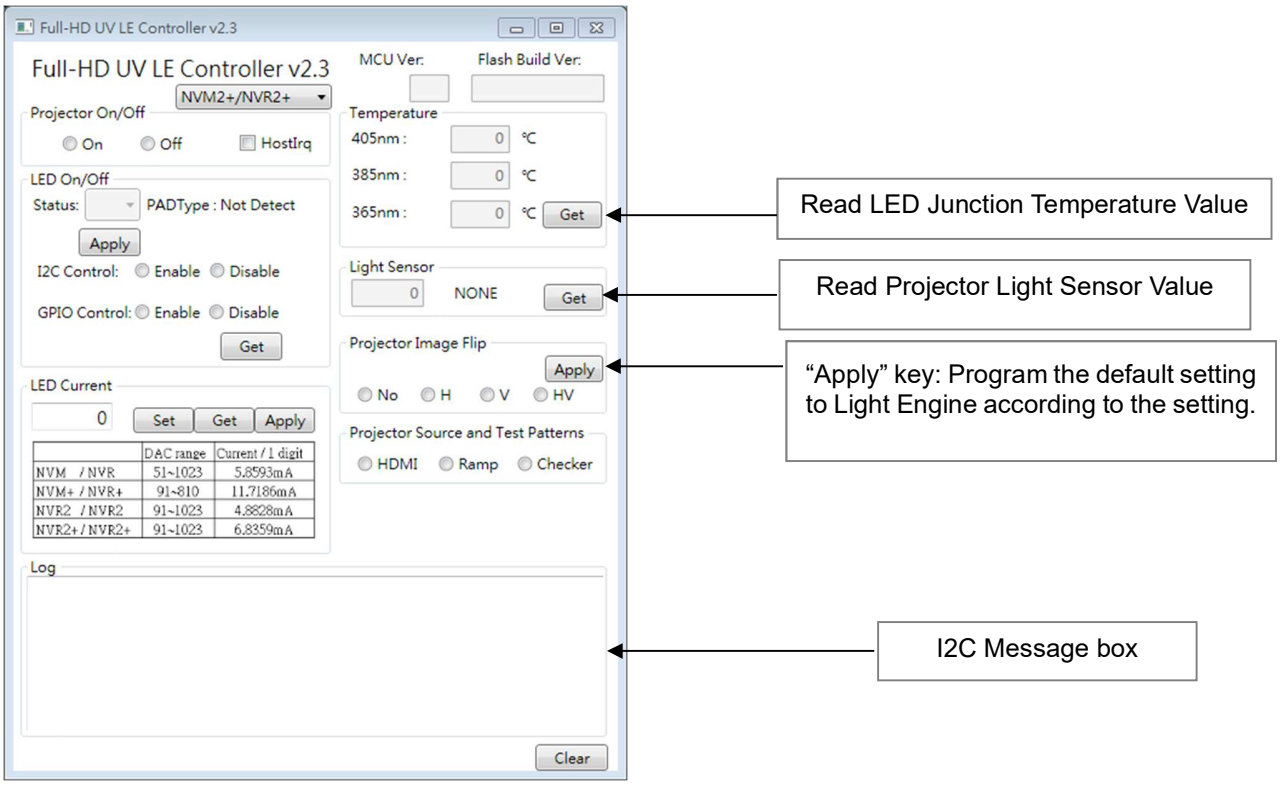
Application 1:



Application 2:



5.7 UI OPERATION INTRODUCTION



"Enable/Disable" Radio Button: For I2C and GPIO LED on / off setting. Both I2C and GPIO Enable, the Led on.

"SET" Key: For LED current setting.

"Get" Key: For LED on / off status, LED immediate current status, LED Junction Temperature status or Light sensor status.

"Apply" Key: Program the default setting to Light Engine according to the setting.

"Projector On/Off": Set light engine on/off

"Temperature" ：Read LED Junction Temperature

"Light Sensor" ：Read Projector Light Sensor value

"Projector Image Flip": Flip projection image

Note:

- We suggest applying thermal solution (ex. fan) before turning on Light Engine to prevent from LED overheating.
- Please make sure USB connection is OK from computer to Light Engine before executing the "LED_Controller.exe"
- LED_Controller folder includes three .dll files (Cypress.dll,cyusbserial.dll and Ddp343x.dll).
- Light Engine default setting

Item	Default Setting
LED Status	Off
Current Default	Setting for 900mW

Video Source	HDMI Input

5.8 I²C COMMAND SETTING

Command format:

Slave device address:0x1B [0x37 (Read) / 0x36 (Write)]

1st Byte = Command

2nd Byte = Data [0]

3rd Byte = Data [1]

(I^2C bus speed：100 kbit/s；I^2C commands conform to the Philips I^2C specification)

5.8.1 LED CURRENT SETTING

Command: 0x54

Data [0] LED current parameter (LSByte)

Data [1] LED current parameter (MSByte)

Data [2] LED current parameter (LSByte)

Data [3] LED current parameter (MSByte)

Data [4] LED current parameter (LSByte)

Data [5] LED current parameter (MSByte)

LED Current Formula=[(Setting value+1)/1024] x 7,

The Max. setting value is 0x3FF (1023).

The Min. LED setting value is 0x5B (91). If the setting value is less than 91 and greater than 0, then the actual setting value will keep at 91.

The step of current setting is 6.8359mA and max. LED current is 7A.

Example of setting value of LED current =7A.

Command: 0x54 // Write LED current

Data[0] = 0xFF // LED current parameter (LSB)

Data[1] = 0x03 // LED current parameter (MSB)

Data[2] = 0xFF // LED current parameter (LSB)

Data[3] = 0x03 // LED current parameter (MSB)

Data[4] = 0xFF // LED current parameter (LSB)

Data[5] = 0x03 // LED current parameter (MSB)

5.8.2 LED CURRENT READING

Step 1: write command

Command: 0x55

Step 2: read data

Data [0] : reserved

Data [1] : reserved

Data [2] : reserved

Data [3] : reserved

Data [4] : LED current parameter (LSByte)

Data [5] : LED current parameter (MSByte)

The transformation from the setting value to the LED Current is just the same as the previous section:

LED_current = [(reading value + 1) / 1024] * 7

= ((Data[5]*256 + Data[4]) + 1) /1024 * 7

5.8.3 LED ENABLE/DISABLE

Command : 0x52 // Write LED Enable / Disable

Data[0] = 0x07 // LED enabled

Command : 0x52 // Write LED Enable / Disable

Data[0] = 0x00 // LED disabled

Note：Please make sure the video signal presented, otherwise the LED will be disable state.

5.8.4 READ LED STATUS

Step 1: write command

Command: 0x53

Step 2: read data

Data[0] : the bit[2] value is mapped to the status of LED disabled/enabled

If (Data[0] & 0x04) == 1, LED is enabled; if it is 0, LED is disabled.

5.8.5 READ LED JUCTION TEMPERATURE

Step 1: write command

Command: 0xD6

Step 2: read data

Data [0] LED Junction Temperature (LSByte)

Data [1] LED Junction Temperature (MSByte)

The two byte data will use only 12 bits , and unused bits will be set 0.

The data is signed in bit11, and 12 bits long.

The bit order and definition for 12bits showed below:

Bit11 : 0 → Positive temperature

1→ Negative temperature

Bit10 to Bit0 : Magnitude of Temperature(Divide by 10 to find Magnitude)

The read out value range is -204.7~204.7

LED Junction Temperature need to be calculated according to LED current status

LED_Junction_Temp = (Data[1]*256 + Data[0])/10 + DW*Vol*LED_I

Note:

- DW is the LED thermal resister is 0.7 (°C/W)
- Vol is the LED operating voltage (V)
- LED_I is the LED current (A)

Vol = -0.0005* LED_I^3+0.0016* LED_I^2+0.3006* LED_I + 6.8986 for 365nm LED

Vol = 0.0008* LED_I^3-0.0266* LED_I^2+0.4798* LED_I + 6.3657 for 385nm LED

Vol = 0.0004* LED_I^3- 0.0292* LED_I^2 + 0.5726*LED_I + 6.3336 for 405nm LED

5.8.6 PROJECTOR IMAGE FLIP SETTING

Command : 0x14 // Write Projector Image Flip Setting

Data[0] = 0x00 // No Flip

Command : 0x14 // Write Projector Image Flip Setting

Data[0] = 0x02 // Horizontal Flip

Command : 0x14 // Write Projector Image Flip Setting

Data[0] = 0x04 // Vertical Flip

Command : 0x14 // Write Projector Image Flip Setting

Data[0] = 0x06 // Horizontal and Vertical Flip

5.8.7 PROJECTOR IMAGE FLIP READING

Step 1: write command

Command: 0x15

Step 2: read data

Data[0] : the bit[2] value is mapped to the status of Vertical Image Flip

the bit[1] value is mapped to the status of Horizontal Image Flip

If (Data[0] & 0x04) == 1, Vertical Flip is enabled; if it is 0, Vertical Flip is disabled.

If (Data[0] & 0x02) == 1, Horizontal Flip is enabled; if it is 0, Horizontal Flip is disabled.

5.8.8 READ LIGHT SENSOR VALUE

The light engine type is TSL25715, it need to use corresponding commands.

Command format :

Light Sensor	Slave address
TSL25715	0x29

1st Byte = Command

2nd Byte = Data [0]

3rd Byte = Data [1]

Step 1: write command

Light Sensor	Command
TSL25715	0xB4

Step 2: read data

Data [0] Light sensor value (LSByte)

Data [1] Light sensor value (MSByte)

Light_Sensor = (Data[1]*256 + Data[0])

Note: Suggest to read the light sensor value with a fixed projected pattern (eg. Black Pattern). Light sensor value may differ with different projected patterns.

Note: Suggest to Check whether Light Sensor is TSL25715 by using LED Controller.

For detailed methods, please refer to our source code of LED Controller.

5.8.9 PROJECTOR SOURCE AND TEST PATTERNS SELECTION

a. HDMI

Command : 0x05 // Projector input source selection

Data[0] = 0x00 // HDMI input source

b. Standby Mode

The Standby mode disables illumination power and sets the DMD in a 50-50 refresh duty cycle, where the mirrors are on 50% of the time and off during the remaining time. This 50-50 refresh state helps in prolonging the life of the DMD.

Command : 0x05 // Projector input source selection

Data[0] = 0xFF // Standby Mode

Note：

Please change the projector source to HDMI mode when returning to normal display mode.

c. Ramp

Command : 0x05 // Projector input source selection

Data[0] = 0x01 // Test Pattern

Command : 0x0b // Pattern type selection

Data[0] = 0x01 //Ramp

Data[1] = 0x70

Data[2] = 0x00

Data[3] = 0xff

Data[4] = 0x00

Data[5] = 0x00

d. Checker

Command : 0x05 // Projector input source selection

Data[0] = 0x01 // Test Pattern

Command : 0x0b // Pattern type selection

Data[0] = 0x07 //Checker

Data[1] = 0x70

Data[2] = 0x04

Data[3] = 0x00

Data[4] = 0x04

Data[5] = 0x00

5.8.10 READ PROJECTOR SOURCE AND TEST PATTERNS SELECTION

Step 1: write command

Command: 0x06

Step 2: read data

Data[0] : the bit[0] value is used to judge if it is HDMI input source, or Test Pattern

If (Data[0] & 0x01) == 0, HDMI input source is selected; if it is 1, Test Pattern is selected.

For the latter case, we have to read another register to distinguish which test pattern is displayed now.

Step 3: write command

Command: 0x0C

Step 4: read data

Data[0] : the least significant 4 bit value is used to judge which test pattern is applied

Data[1] : reserved

Data[2] : reserved

Data[3] : reserved

Data[4] : reserved

Data[5] : reserved

If (Data[0] & 0x0F) == 0x01, *Ramp* is selected; if it is 0x07, *Checker* is selected.

5.9 CYPRESS API FUNCTION INTRODUCTION

There are different versions of Cypress SDK libraries for Windows and Linux; even in Windows, it is also different between the two programming languages C++ and C# when we use to load Cypress SDK library, so we recommend our customers to check Cypress Semiconductor website for the details.

Here we introduce some fundamental functions to be used in all kinds of the above development environments.

Here we introduce some fundamental functions to be used in all kinds of the above development environments.

5.9.1 DEVICE-INDEPENDENT FUNCTIONS

We use Cypress chip (CY7C65215) to communicate with the other devices via the GPIO, I2C and SPI interfaces. However, no matter which interface we use, the following three API functions are all necessary.

5.9.1.1 CyGetDeviceInfoVidPid

CY_RETURN_STATUS CyGetDeviceInfoVidPid (vidPid, *deviceIdList, *deviceInfoList, *deviceCount, infoListLength)

Input:

parameter	data type	description
vidPid	CY_VID_PID	VID and PID of device of interest
deviceIdList	UINT8	Array of device ID's returned
deviceInfoList	CY_DEVICE_INFO	Array of pointers to device info list
deviceCount	UINT8	Count of devices with specified VID PID
infoListLength	UINT8	Total length of the deviceInfoList allocated (Size of deviceInfoList array)

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_DEVICE_INFO_FETCH_FAILED	Device information fetch failed
CY_ERROR_ACCESS_DENIED	Access of the API is denied for the application
CY_ERROR_DEVICE_NOT_FOUND	Device disconnected

Description:

For a given VID and PID, the API returns deviceIdList and deviceInfoList.

Note:

The deviceIdList contains the device numbers of all the devices with specified VID and PID.

Parameter	Value
USB Vendor ID (VID)	0x04B4
USB Product ID (PID)	0x000A

Using deviceInfoList application can identify the device of interest. Information that is provided includes interface number, string descriptor, deviceBlock, deviceType and deviceClass. We have to pick up the device whose deviceBlock matches the specific SCB number (0 for GPIO & I2C, 1 for SPI) to use its index in deviceInfoList as the deviceNumber parameter of the CyOpen.

Serial Communication Block (SCB) Usage	
SCB0	I2C
SCB1	SPI

5.9.1.2 CyOpen

CY_RETURN_STATUS CyOpen (deviceNumber, interfaceNum, *handle)

Input:

parameter	data type	description
deviceNumber	UINT8	Device number of device that needs to be opened
interfaceNum	UINT8	Interface number
handle	CY_HANDLE	Handle returned by the API

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_DRIVER_OPEN_FAILED	Failed to open a device in the library
CY_ERROR_ACCESS_DENIED	Access of the API is denied for the application
CY_ERROR_ALLOCATION_FAILED	Error in Allocation of the resource inside the library
CY_ERROR_DEVICE_NOT_FOUND	Device disconnected

Description:

This API is used to open an USB Serial device based on the device number.

Note:

The 2nd argument interfaceNum is used on Linux and Mac OS while obtaining handle for specific interface. In Windows family of operating system, this argument should be set to zero.

5.9.1.3 CyClose

CY_RETURN_STATUS CyClose (handle)

Input:

parameter	data type	description
handle	CY_HANDLE	Handle of the device that needs to be closed

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed

Description:

This API closes the device handle and releases all the resources allocated internally in the library. This API should be invoked using a valid device handle and upon successful return of CyOpen.

5.9.2 GPIO

There are two API functions specific for the GPIO interface.

5.9.2.1 CySetGpioValue

CY_RETURN_STATUS CySetGpioValue (handle, gpioNumber, value)

Input:

parameter	data type	description
handle	CY_HANDLE	Valid device handle
gpioNumber	UINT8	GPIO number[1]
value	UINT8	Value that needs to be set

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_IO_TIMEOUT	Timed out while processing a user request
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed

Description:

This API is used to set the value of a GPIO. It can only set the value of a GPIO that is configured as an output.

5.9.2.2 CyGetGpioValue

CY_RETURN_STATUS CyGetGpioValue (handle, gpioNumber, *value)

Input:

parameter	data type	description
handle	CY_HANDLE	Valid device handle
gpioNumber	UINT8	GPIO number[1]
value	UINT8	Current state of the GPIO

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_IO_TIMEOUT	Timed out while processing a user request
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed

Description:

This API retrieves the value of a GPIO.

[1] The indexes of the GPIO pins that we have to read are only 1 (I2C_BUSY) and 15 (ASIC_READY). The indexes of the GPIO pins that we have to write are only 0 (RESETZ) and 14 (LED_ON_OFF).

5.9.2.3 Function Implementation (GPIO)

There are several GPIO pins prepared in Cypress CY7C65215, from GPIO_0 to GPIO_18, but we only use the two GPIO pins specified as below：

index	name
GPIO_2	PROJ_ON
GPIO_5	HOST_IRQ

5.9.2.3.1 Projector On/Off (Set)

Set GPIO-2(PROJ_ON) to 1 to turn on the light engine(controller).

Set GPIO-2(PROJ_ON) to 0 to turn off the light engine(controller). It won't power off the light engine but set those major components into a safe state, so we ***must*** clear it off before shutdown.

5.9.2.3.2 HOST_IRQ Signal (Get)

Host_IRQ signal indicate when DPP343x auto-initialization is in progress and most importantly when it completes.

When GPIO-5(HOST_IRQ) is pulled high (1), this indicates that the ASIC is performing boot-up and auto-initialization.

Upon completion of Auto-Init, SW will set GPIO 5(HOST_IRQ) to a logic Low(0) state to indicate the completion of Auto-Init.

5.9.3 I2C

There are two API functions specific for the I2C interface.

5.9.3.1 CyI2cWrite

CY_RETURN_STATUS CyI2cWrite (handle, *dataConfig, *writeBuffer, timeout)

Input:

parameter	data type	description
handle	CY_HANDLE	Valid device handle
dataConfig	CY_I2C_DATA_CONFIG	I2C data config
writeBuffer	CY_DATA_BUFFER	Write buffer details
timeout	UINT32	API timeout value

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed
CY_ERROR_IO_TIMEOUT	Timed out while processing a user request
CY_ERROR_PIPE_HALTED	Pipe halted while trying to transfer data
CY_ERROR_DEVICE_NOT_FOUND	Device disconnected
CY_ERROR_BUFFER_OVERFLOW	OverFlow of buffer while trying to read/write data
CY_ERROR_ALLOCATION_FAILED	Error in Allocation of the resource inside the library
CY_ERROR_I2C_DEVICE_BUSY	I2C device busy
CY_ERROR_I2C_NAK_ERROR	I2C device NAK
CY_ERROR_I2C_ARBITRATION_ERROR	I2C bus arbitration error
CY_ERROR_I2C_BUS_ERROR	I2C bus error
CY_ERROR_I2C_STOP_BIT_SET	I2C master has sent a stop bit during a transaction

Description:

This API provides an interface to write data to the I2C device connected to USB Serial.

Note:

The writeBuffer parameter needs to be initialized with buffer pointer, number of bytes to be written before invoking the API. On return, transferCount field contains number of bytes actually written to the device.

CY_I2C_DATA_CONFIG structure specifies parameter such as setting stop bit, Nak and slave address of the I2C device being communicated when USB Serial is master.

5.9.3.2 CyI2cRead

CY_RETURN_STATUS CyI2cRead (handle, *dataConfig, *readBuffer, timeout)

Input:

parameter	data type	description
handle	CY_HANDLE	Valid device handle
dataConfig	CY_I2C_DATA_CONFIG	I2C data config
readBuffer	CY_DATA_BUFFER	Read buffer details
timeout	UINT32	API timeout value

Return:

name	description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed
CY_ERROR_IO_TIMEOUT	Timed out while processing a user request
CY_ERROR_DEVICE_NOT_FOUND	Device disconnected
CY_ERROR_BUFFER_OVERFLOW	OverFlow of buffer while trying to read/write data
CY_ERROR_ALLOCATION_FAILED	Error in Allocation of the resource inside the library
CY_ERROR_I2C_DEVICE_BUSY	I2C device busy
CY_ERROR_I2C_NAK_ERROR	I2C device NAK
CY_ERROR_I2C_ARBITRATION_ERROR	I2C bus arbitration error
CY_ERROR_I2C_BUS_ERROR	I2C bus error
CY_ERROR_I2C_STOP_BIT_SET	I2C master has sent a stop bit during a transaction

Description:

This API provides an interface to read data from the I2C device connected to USB Serial.

Note:

The readBuffer parameter needs to be initialized with buffer pointer, number of bytes to be read before invoking the API. On return, the transferCount field will contain the number of bytes read back from device.

CY_I2C_DATA_CONFIG structure specifies parameters such as setting stop bit, Nak and slave address of the I2C device.

5.9.4 SPI

There is one API function specific for the SPI interface.

5.9.4.1 CySpiReadWrite

CY_RETURN_STATUS CySpiReadWrite (handle, *readBuffer, *writeBuffer, timeout)

Input:

parameter	data type	description
handle	CY_HANDLE	Valid device handle
readBuffer	CY_DATA_BUFFER	Read data buffer
writeBuffer	CY_DATA_BUFFER	Write data buffer
timeout	UINT32	Time out value of the API

Return:

name	Description
CY_SUCCESS	API returned successfully without any error
CY_ERROR_INVALID_HANDLE	Device handle is invalid
CY_ERROR_INVALID_PARAMETER	One or more parameters sent to the API was invalid
CY_ERROR_REQUEST_FAILED	Request sent to USB Serial device failed
CY_ERROR_IO_TIMEOUT	Timed out while processing a user request
CY_ERROR_PIPE_HALTED	Pipe halted while trying to transfer data
CY_ERROR_DEVICE_NOT_FOUND	Device disconnected
CY_ERROR_BUFFER_OVERFLOW	OverFlow of buffer while trying to read/write data

Description:

This API provides an interface to do data transfer with the SPI slave/master connected to USB Serial device.

Note:

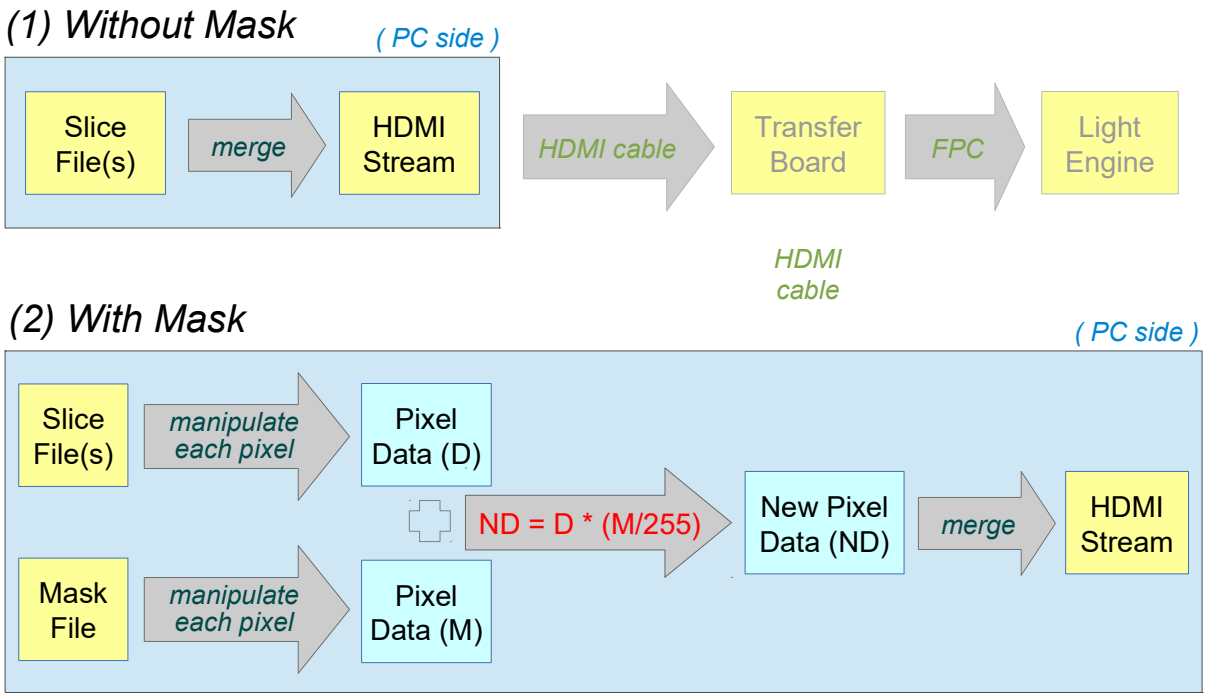
To perform read only operation, pass NULL as argument for writeBuffer and to perform write only operation pass NULL as an argument for readBuffer. On return, the transferCount field will contain the number of bytes read and/or written.

5.10 PIXEL DATA MIXED WITH MASK FILE AND READ MASK FILE FROM SPI FLASH

5.10.1 PREREQUISITE

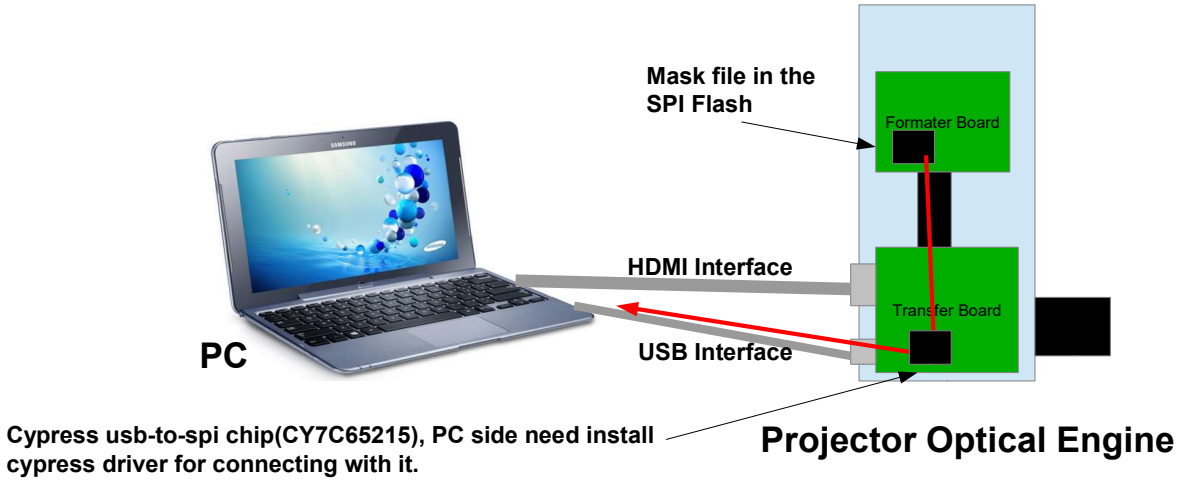
- The mask file is a 8-bit PNG file, with the resolution of 1920*1080, and each pixel data would be a value of 0~255.
- The mask file must be pre-mixed with each slice file in advance, on the PC side, and pixel-by-pixel.
- The mixed method is (New Pixel Data) = (Original Pixel Data) * ((Mask Data) / 255)

5.10.2 MIXED OPERATION WITH MASK FILE

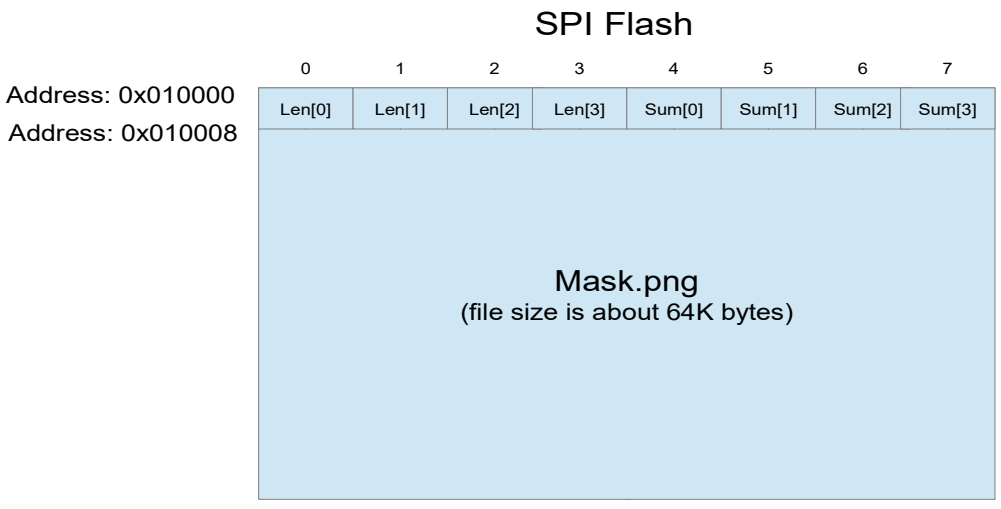


5.10.3 MASK FILE READ FROM SPI FLASH

- The Mask file can be read by PC software through USB interface (shown as red line path)



5.10.4 SPI FLASH DATA STRUCTURE FOR MASK FILE

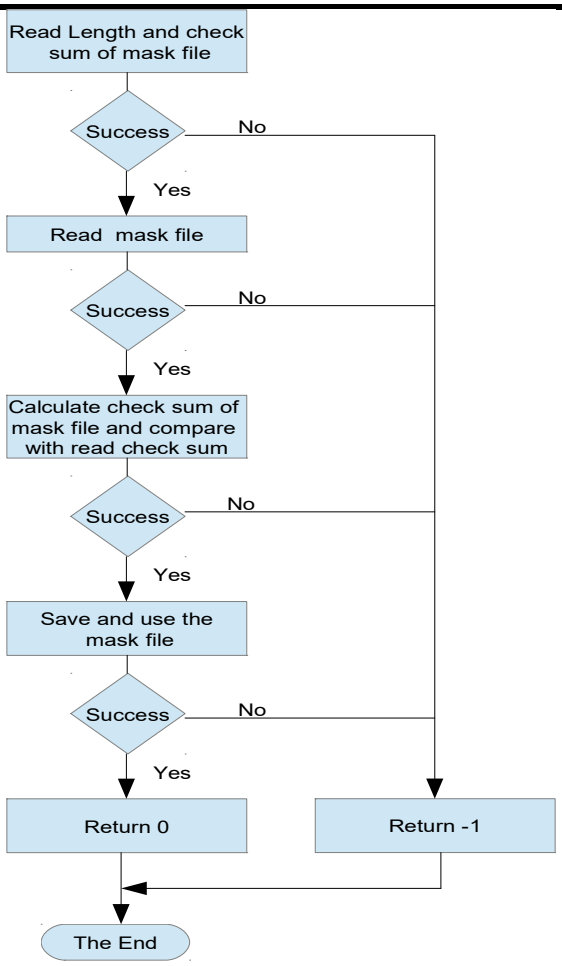


Address: 0x010000~0x010003 → The length of mask file

Address: 0x010004~0x010007 → The check sum of mask file

Address: 0x010008~ → The mask file (file size is about 64K bytes)

5.10.5 Read Mask file steps



Please refer to “read mask sample code written in C++” for detail.

5.10.6 Read calibrated LED current and light sensor value

The light output power of UV engine is calibrated according to the specification. The correspondence LED current and light sensor value is stored in the SPI Flash. The reading method of the LED current and light sensor is similar to reading mask file shown in 5.10.5.

Calibrated LED current is read from address 43 (0x2B) and the length is four bytes.

Calibrated LED current = {byte3,byte2,byte1,byte0} x 4.8828 / 1000 Amp.

Light Sensor value is read from address 51(0x33) and the length is four bytes.

Light Sensor value = {byte3,byte2,byte1,byte0}.

5.11 FOCUS MOTOR SPECIFICATION

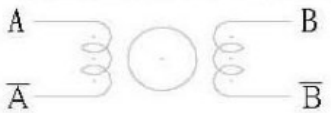
(a) motor specification：

Standard Operating Conditions	
Rated Voltage	3.3V DC
Operating Voltage Range	3.3V～4.5V
Excitation	2 Phase Excitation
Drive Mode	Bi-Polar Drive

Electrical Specification	
Coil Resistance	21Ω±7%
Step Angle	0.2687/STEP
Withstand Voltage	AC 100V/3mmA(for 1 second between case and coil)
Insulation Resistance	1.0M(min.) (Between case and coil DC 100V)

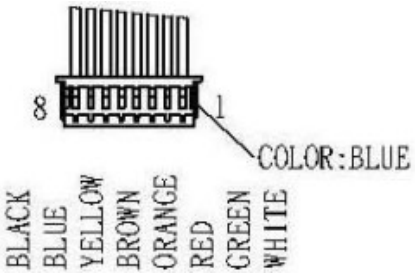
PIN	COLOR	PHASE
8	BLACK	$\overline{A}$ REAR
7	BLUE	A REAR
6	YELLOW	$\overline{A}$ FRONT
5	BROWN	A FRONT
4	ORANGE	$\overline{B}$ FRONT
3	RED	B FRONT
2	GREEN	B REAR
1	WHITE	$\overline{B}$ REAR

CONNECTION DIAGRAM



SEQUENCE OF EXCITATION

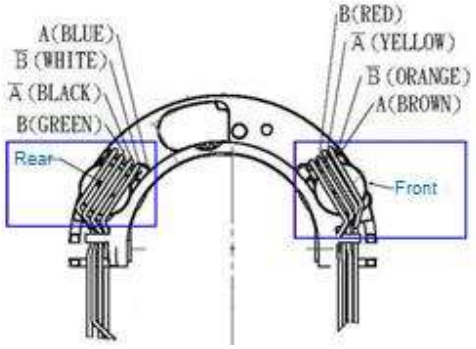
STEP / PHASE	1	2	3	4
A	H	H	L	L
$\overline{A}$	L	L	H	H
B	L	H	H	L
$\overline{B}$	H	L	L	H



Connector Type: Molex 51021-0800

(b) Motor of projection lens：

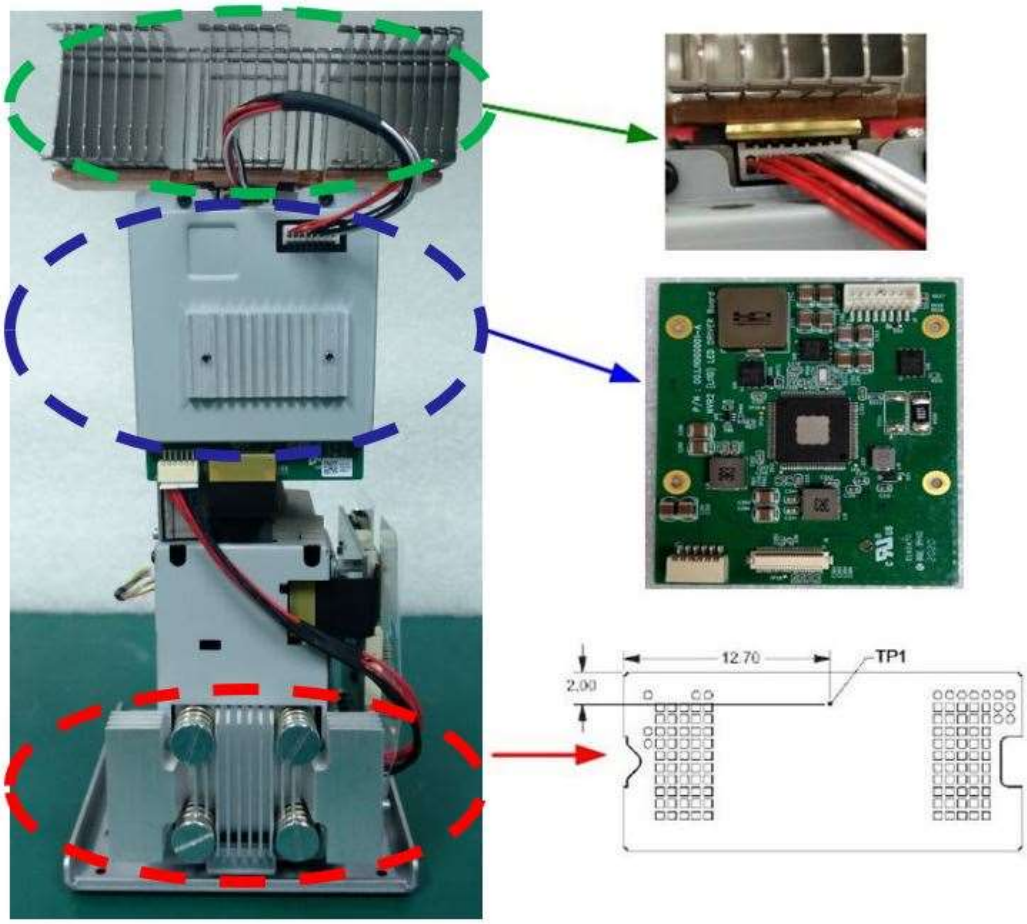
1. Front motor for adjust the Front Group of projection lens.
2. Rear motor for adjust the Rear Group of projection lens.



6 MAX. THERMAL RECOMMENDATION

Please cool down the temperature of the heat-sinks of LED, LED driver (PAD3005), DMD. under the limitation illustrated in below pictures to prevent the permanent damage of devices.

To prevent damaging the component of light engine, the LED junction temperature should be kept under 125℃, temperature of heat sink of LED driver board should be kept under 100℃.



7 PACKING

Antistatic package

8 TROUBLE SHOOTING

(1) If the light engine can not be turn on or no image projected, please turn off the light engine. Then, check the cable connections and turn it on again.

(2) If you can not control the light engine via the software, please re-connect the USB cable or check if the software is installed correctly.